POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 1

BUSINESS INTELLIGENCE REPORT

Project: BI Analysis of E-Commerce Transactions

Group: 9

Student: B22DCAT184 - Lê Đức Mạnh

B22DCVT230 - Trịnh Đăng Hùng

Class: E22HTTT

Project Report - Brazilian E-Commerce (Olist Dataset)

Tools: Python - MySQL - Metabase

Table of Contents

1. Introduction

1.1 Business Context

What is the problem we are solving?

Olist is a Brazilian e-commerce marketplace that operates by connecting thousands of small and medium enterprises (SMEs) with major e-commerce platforms in the country, such as Mercado Livre and B2W. Instead of operating their own online stores - which requires significant technology and marketing expenses - sellers register with Olist to reach millions of customers through a single account.

Between September 2016 and October 2018, Olist processed over **99,000 orders** spanning all **27 states** of Brazil. The operational system generated multiple separate data streams: order information, product details within orders, payment history, customer reviews, geographical locations of buyers and sellers, etc. All of these streams were stored in separate operational databases (OLTP) that were not designed for analytical purposes.

The core problem: **data exists but cannot be used for business decisions.** A simple question like *"Which product category generated the most revenue this month?"* requires JOINing 4–5 different tables and writing dozens of SQL lines - a task only developers could do, and which often took hours.

Why does this problem need Business Intelligence?

The gap between **raw data** and **business decisions** is a challenge that every e-commerce company faces. For Olist, this gap is particularly evident because:

- **Fragmented Data:** 9 separate CSV files/tables, each recording only one aspect of an order. No single table has enough information to answer business questions directly.
- **Decision Makers Are Not Engineers:** Business directors, marketing heads, and logistics managers cannot and should not have to write SQL every time they need a number.
- **Real-Time is a Competitive Factor:** In e-commerce, slow reactions to data (e.g., not knowing that revenue is declining) directly result in lost revenue.

A **Data Warehouse + BI Dashboard** system solves this problem by:

- **Pre-joining and pre-aggregating** all data into a Star Schema structure optimized for analysis - instead of JOINing every time a query is run.
- **Self-service analytics:** Metabase allows managers to drag-and-drop to create charts without writing code.
- **Single source of truth:** All departments look at the same cleaned and standardized numbers.
- **Automated metrics:** Metrics like delivery_days, is_on_time, and delay_days are pre-calculated in the ETL - analysts do not need to calculate them manually.

1.2 Business Questions We Want to Answer

This project is designed to answer **5 strategic business question groups** through 4 dashboards in Metabase:

#	**Business Question**	**Why It Matters**	**Dashboard Section**
1	Which product category generates the most revenue?	Prioritize marketing budget allocation	Product Analysis
2	How does revenue fluctuate by month/quarter?	Detect growth trends & seasonality	Executive KPI Overview
3	Do customers purchase multiple times or just once?	Evaluate retention effectiveness	Customer Behavior
4	Which state/region contributes the most revenue?	Direct geographical market expansion	Geographic Analysis
5	How does late delivery affect review scores?	Improve SLA & customer experience	Operations & Delivery

Methodology Note: Each business question above maps directly to one or more SQL queries and charts in Metabase. The results are not just numbers - each insight comes with a specific **Business Action**.

1.3 Project Scope & Tools

This project implements a **complete end-to-end data pipeline**, including:

Component	Tool Used	Why This Choice
Raw Data Storage	CSV files (9 files)	Original format of the Olist dataset from Kaggle
Staging Database	MySQL - olist_staging	Exact replica of the CSVs; allows restarting ETL without re-reading files
Data Warehouse	MySQL - olist_dwh	Star Schema optimized for OLAP queries in Metabase
ETL Language	Python 3.10 + pandas 2.x	Flexible, handles complex data cleaning well
DB Connector	SQLAlchemy + PyMySQL	Standard ORM, supports fast bulk inserts
BI Dashboard	Metabase (self-hosted)	Free, connects directly to MySQL, drag-and-drop
Logging	Python logging module	Logs each ETL step for debugging and auditing

Data Scope:

- **Timeframe:** September 2016 - October 2018 (approx. 25 months)
- **Number of Orders:** ~99,441 orders
- **Number of Line Items:** ~112,650 order items (grain of the fact table)
- **Geography:** 27 Brazilian states, 5 macro-regions
- **Out of Scope:** Does not include real-time data, inventory tracking, or marketing attribution models.

2. Dataset Description

2.1 Data Source

Attribute	Details
Dataset Name	Brazilian E-Commerce Public Dataset by Olist
Source	Kaggle - olistbr/brazilian-ecommerce
Period	04/09/2016 - 17/10/2018 (25 months 13 days)
License	CC BY-NC-SA 4.0 (academic research only, non-commercial)
Characteristics	Real operational data from Olist platform, anonymized
Language	Portuguese (city names, product categories) → translated to English

Why is this dataset suitable for learning BI? Olist is the most "comprehensive" e-commerce dataset publicly available - it includes the entire order lifecycle: from ordering → payment → delivery → customer reviews. Most other public datasets only have 1–2 tables, which does not reflect real-world complexity.

2.2 Files Overview - Actual Statistics

The dataset consists of **9 CSV files** storing different aspects of business operations. Below are the actual measurements profiled from the data:

#	File	Description	Row Count (Actual)	Column Count	Has NULLs?
1	olist_orders_dataset.csv	Each order - main "receipt"	**99,441**	8	Yes
2	olist_order_items_dataset.csv	Individual items per order	**112,650**	7	No
3	olist_order_payments_dataset.csv	Payment transactions	**103,886**	5	No
4	olist_order_reviews_dataset.csv	Customer reviews	**99,224**	7	Yes
5	olist_customers_dataset.csv	Customer geographic info	**99,441**	5	No
6	olist_sellers_dataset.csv	Seller geographic info	**3,095**	4	No
7	olist_products_dataset.csv	Product categories & dimensions	**32,951**	9	Yes
8	olist_geolocation_dataset.csv	ZIP prefix → GPS coordinates	**1,000,163**	5	No
9	product_category_name_translation.csv	PT → EN category name translation	**71**	2	No

Why does order_items have more rows than orders? An order can contain multiple items from different sellers. For example: order #001 buys 3 products → 1 row in orders, 3 rows in order_items. Total rows: 99,441 orders → 112,650 line items, averaging **1.14 products per order**.

2.3 Detailed File Analysis - Column-Level Analysis

File 1: olist_orders_dataset.csv - 99,441 rows × 8 columns

This is the **central table** - all other tables join through order_id or customer_id.

Column	Data Type	Null Count (Actual)	Description	ETL Note
order_id	VARCHAR(32)	0 (0.0%)	Unique order identifier - **Primary Key**	Keep as-is
customer_id	VARCHAR(32)	0 (0.0%)	FK → olist_customers	Keep as-is
order_status	VARCHAR(20)	0 (0.0%)	Order status (8 values)	Keep as-is
order_purchase_timestamp	DATETIME	0 (0.0%)	Order purchase timestamp - minimum time	Used to build dim_time
order_approved_at	DATETIME	**160 (0.2%)**	Timestamp when payment is approved	NULL = not approved yet
order_delivered_carrier_date	DATETIME	**1,783 (1.8%)**	Handover timestamp to the carrier	NULL = not shipped yet
order_delivered_customer_date	DATETIME	**2,965 (3.0%)**	Handover timestamp to the customer - **most important**	NULL = not delivered yet

Column	Data Type	Null Count (Actual)	Description	ETL Note
order_estimated_delivery_date	DATETIME	0 (0.0%)	Promised estimated delivery date	Used to calculate delay_days

Distribution of order_status (Actual):

Status	Volume	Proportion	Meaning
delivered	96,478	**97.0%**	Successfully delivered → primary analysis focus
shipped	1,107	1.1%	In transit
canceled	625	0.6%	Canceled
unavailable	609	0.6%	Out of stock
invoiced	314	0.3%	Invoiced, awaiting shipment
processing	301	0.3%	Processing
Other	7	0.0%	created, approved

In our dashboard analysis, we filter WHERE order_status = 'delivered' to ensure we only calculate revenue from fully completed transactions. **97% of orders are delivered**, which is a very high delivery success rate, indicating that Olist operates efficiently.

File 2: olist_order_items_dataset.csv - 112,650 rows × 7 columns

Grain of the fact table - This is the core table of the entire Data Warehouse.

Column	Data Type	Null Count	Description
order_id	VARCHAR(32)	0	FK → olist_orders
order_item_id	INT	0	Sequential number of item within order (1, 2, 3, ...)
product_id	VARCHAR(32)	0	FK → olist_products
seller_id	VARCHAR(32)	0	FK → olist_sellers
shipping_limit_date	DATETIME	0	Seller shipping deadline
price	FLOAT	0	Product price (R$) - primary measure
freight_value	FLOAT	0	Shipping/Freight cost (R$)

Price Statistics (Actual):

Metric	Value
Minimum Price	R$ 0.85
Maximum Price	R$ 6,735.00
Average Price (mean)	R$ 120.65
Median Price (median)	R$ 74.99
Q1 (25th percentile)	R$ 39.90
Q3 (75th percentile)	R$ 134.90
P90 (90th percentile)	R$ 229.80

Strong right-skewed price distribution: The median (R$ 74.99) is much lower than the mean (R$ 120.65), showing that a few very high-priced products pull the average up. In analysis, use the **median** to understand the "typical price" and the **mean** when calculating total revenue.

Freight Value: min=R$0.00, max=R$409.68, avg=R$19.99

Multi-item Orders: 9,803 orders contain 2 or more products (accounting for ~10% of total orders).

File 3: olist_order_payments_dataset.csv - 103,886 rows × 5 columns

ETL Challenge: An order can have multiple payment rows (when the customer combines credit card + voucher).

Column	Data Type	Null Count	Description
order_id	VARCHAR(32)	0	FK → olist_orders (NOT UNIQUE)
payment_sequential	INT	0	Sequential sequence of payment methods
payment_type	VARCHAR(20)	0	Payment method
payment_installments	INT	0	Number of installments (1 = lump sum, max=24)
payment_value	FLOAT	0	Paid amount (R$)

Payment Type Distribution (Actual):

Method	Transactions	Proportion
credit_card	76,795	**73.9%**
boleto	19,784	19.0%

Method	Transactions	Proportion
voucher	5,775	5.6%
debit_card	1,529	1.5%
not_defined	3	0.0%

> **Boleto** is a very popular payment method in Brazil - similar to a bank slip, customers print it out and pay at an ATM or a bank branch. The 19% boleto rate reflects the unique nature of the Brazilian market where not everyone has a credit card.

ETL Handling of Multi-payments: 2,961 orders have >1 payment rows → must GROUP BY order_id with SUM(payment_value) before joining into the fact table.

File 4: olist_order_reviews_dataset.csv - 99,224 rows × 7 columns

Column	Data Type	Null Count	Description
review_id	VARCHAR(32)	0	Primary Key of the review
order_id	VARCHAR(32)	0	FK → olist_orders
review_score	FLOAT	0	Review score: 1–5
review_comment_title	VARCHAR	**87,656 (88.3%)**	Review title (mostly empty)
review_comment_message	VARCHAR	**58,247 (58.7%)**	Review text comment content
review_creation_date	DATETIME	0	Creation date of review
review_answer_timestamp	DATETIME	0	Handle timestamp of review response

Review Score Distribution (Actual):

Score	Reviews	Proportion	Classification
⭐⭐⭐⭐⭐ 5	57,328	**57.8%**	Very Satisfied
⭐⭐⭐⭐ 4	19,142	19.3%	Satisfied
⭐⭐⭐ 3	8,179	8.2%	Neutral
⭐⭐ 2	3,151	3.2%	Dissatisfied
⭐ 1	11,424	11.5%	Very Dissatisfied
Average	**4.09 / 5.00**		

> **1-star reviews are higher than 2-star reviews (11.5% > 3.2%)** - this is an interesting phenomenon. In e-commerce, dissatisfied customers tend to give the lowest possible score (1 star) rather than a moderate low score. This forms a "bimodal" distribution (two peaks: highly satisfied and highly dissatisfied).

File 5: olist_customers_dataset.csv - 99,441 rows × 5 columns

Column	Data Type	Null Count	Notes
customer_id	VARCHAR(32)	0	PK - unique for EACH ORDER (not each customer)
customer_unique_id	VARCHAR(32)	0	Real unique buyer ID (96,096 unique - fewer than customer_id)
customer_zip_code_prefix	VARCHAR(10)	0	First 5 digits of customer ZIP code
customer_city	VARCHAR(60)	0	City name (not normalized)
customer_state	VARCHAR(2)	0	State code (2 letters)

Important: customer_id ≠ real unique customer! Olist generates a new customer_id for **every order**. customer_unique_id is the actual unique identifier of the customer. There are 99,441 customer_id values but only 96,096 customer_unique_id values → meaning about 3,345 customers purchased ≥2 times. Our ETL joins using customer_id to maintain referential integrity with the orders table.

Top 5 States with Most Customers:

State	Name	Customer Count	Proportion
SP	São Paulo	41,746	**42.0%**
RJ	Rio de Janeiro	12,852	12.9%
MG	Minas Gerais	11,635	11.7%
RS	Rio Grande do Sul	5,466	5.5%
PR	Paraná	5,045	5.1%

File 6: olist_sellers_dataset.csv - 3,095 rows × 4 columns

Smallest dataset (excluding translation table). No NULLs, no duplicates.

Column	Data Type	Null Count	Notes
seller_id	VARCHAR(32)	0	Primary Key - 3,095 unique sellers
seller_zip_code_prefix	VARCHAR(10)	0	First 5 digits of seller ZIP code
seller_city	VARCHAR(60)	0	City name (611 unique cities)
seller_state	VARCHAR(2)	0	State code (23/27 states have sellers)

File 7: olist_products_dataset.csv - 32,951 rows × 9 columns

Column	Data Type	Null Count (Actual)	ETL Notes
product_id	VARCHAR(32)	0	Primary Key
product_category_name	VARCHAR	**610 (1.9%)**	Portuguese category name → needs translation
product_name_lenght	FLOAT	610 (1.9%)	Product name character length
product_description_lenght	FLOAT	610 (1.9%)	Description character length
product_photos_qty	FLOAT	610 (1.9%)	Number of product photos
product_weight_g	FLOAT	**2 (0.0%)**	Product weight (grams)
product_length_cm	FLOAT	2 (0.0%)	Length (cm)
product_height_cm	FLOAT	2 (0.0%)	Height (cm)
product_width_cm	FLOAT	2 (0.0%)	Width (cm)

610 products (1.9%) lack category names → ETL fills them with "unknown". There are **73 categories** in total, of which the translation table contains 71 (the 2 categories missing translations keep their original Portuguese names).

File 8: olist_geolocation_dataset.csv - 1,000,163 rows × 5 columns

Largest file (~61 MB), mapping ZIP codes to GPS coordinates. It contains 261,831 duplicate rows (same ZIP, different coordinates - multiple GPS points for one ZIP area).

ETL Processing: Sample 20% (200,032 rows) with random_state=42 to reduce loading time while maintaining representativeness. This file is for reference only - primary geographic analysis uses customer_state and seller_state.

File 9: product_category_name_translation.csv - 71 rows × 2 columns

Smallest table - pure lookup table, no NULLs, no duplicates.

product_category_name (PT) →product_category_name_english (EN)

"beleza_saude"→"health_beauty"

"cama_mesa_banho"→"bed_bath_table"

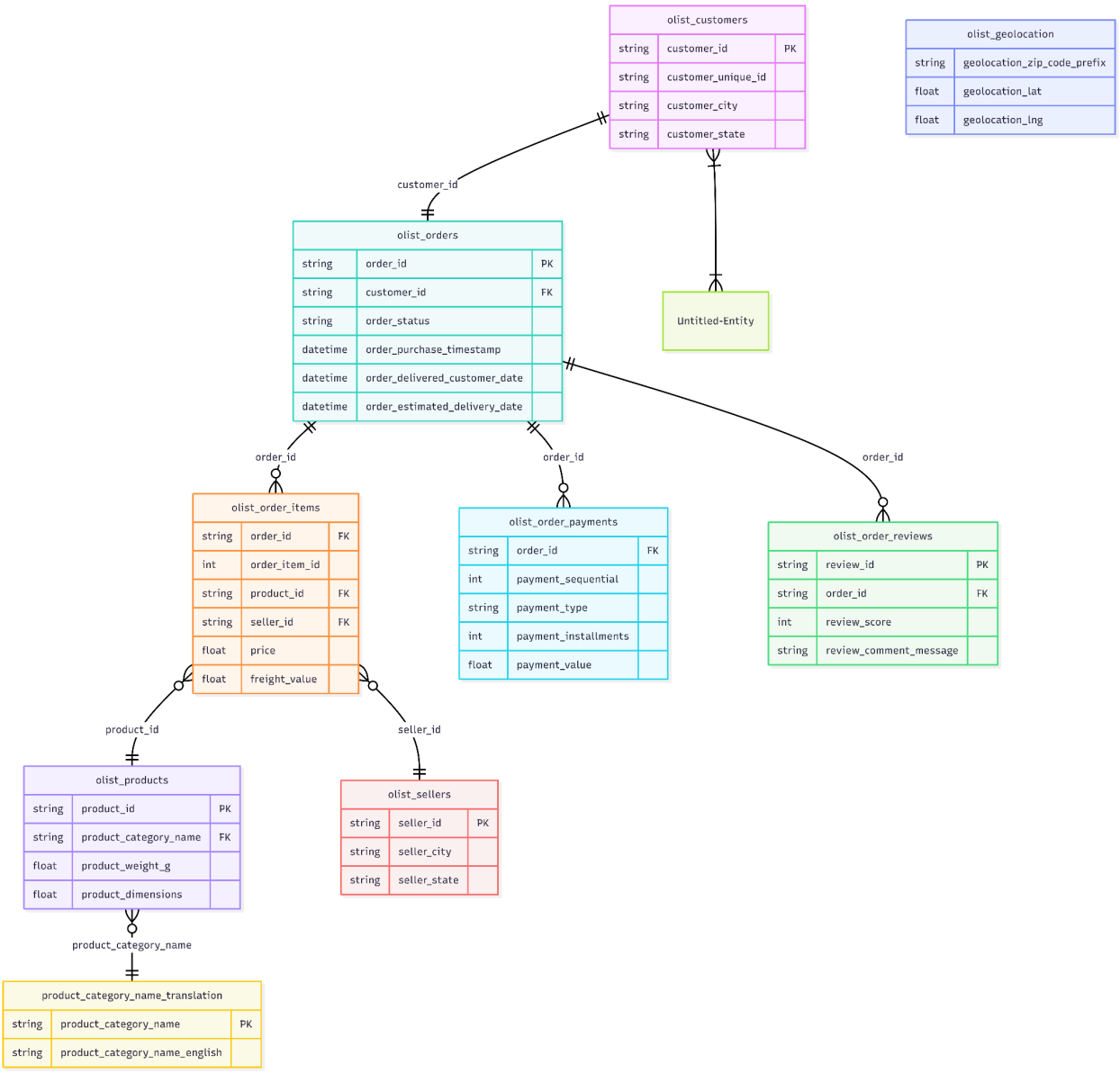
"eletronicos"→ "electronics"

... (71 rows)

ETL: LEFT JOIN with olist_products - products without translations retain their original Portuguese names.

2.4 How the Files Relate to Each Other (Entity Relationship)

Full entity relationship schema showing foreign keys across the 9 files:



(Used as reference, not joined directly to the fact table)

Why does olist_order_payments have more rows than olist_orders? A single order can use multiple payment methods at once. For example:

order_id	payment_type	payment_value	
order_001	credit_card	R$ 80.00	← row 1
order_001	voucher	R$ 20.00	← row 2 (same order)

Total: 99,440 unique orders but 103,886 payment records. ETL must GROUP BY order_id → SUM(payment_value) before joining.

2.5 Data Quality Profile - Actual Profiling Results

Prior to building the Data Warehouse, the entire dataset was audited via profiling scripts (etl/01_extract.py). Below are the actual findings:

Data Quality Overview

File	Total Rows	Duplicate Rows	Columns with NULLs	Total NULL Cells	Assessment
orders	99,441	0	3	4,908	Expected NULLs
order_items	112,650	0	0	0	Perfectly clean
payments	103,886	0	0	0	Perfectly clean
reviews	99,224	0	2	145,903	High number of empty comments
customers	99,441	0	0	0	Perfectly clean
sellers	3,095	0	0	0	Perfectly clean

File	Total Rows	Duplicate Rows	Columns with NULLs	Total NULL Cells	Assessment
products	32,951	0	8	2,448	1.9% missing categories
geolocation	1,000,163	**261,831**	0	0	26% duplicate ZIPs
translation	71	0	0	0	Perfectly clean

Discovered Issues & ETL Solutions

#	Dataset	Discovered Issue	Severity	ETL Mitigation
1	orders	2,965 rows (3.0%) order_delivered_customer_date are NULL	Normal	Retained NULLs - order not yet delivered, is_on_time = NULL
2	orders	160 rows (0.2%) order_approved_at are NULL	Low	Retained NULLs - order not yet approved
3	order_items	No NULLs, no duplicates	OK	No action required
4	payments	2,961 orders have multiple payment rows	Common	groupby('order_id').agg({'payment_value':'sum', 'payment_type':'first'})
5	reviews	58,274 (58.7%) review_comment_message are empty	Normal	Acceptable - reviews are optional, not mandatory

#	Dataset	Discovered Issue	Severity	ETL Mitigation
6	reviews	87,656 (88.3%) review_comment_title are empty	Normal	Acceptable - unused in analysis
7	customers	Inconsistent city names ("SÃO PAULO", "são paulo")	Common	str.strip().str.lower().str.title() → "Sao Paulo"
8	products	610 (1.9%) missing product_category_name	Low	Filled with "unknown"
9	products	2 products missing weight/dimensions	Very Low	Filled with 0
10	sellers	Inconsistent city names	Common	str.strip().str.lower().str.title()
11	geolocation	261,831 duplicate rows (same ZIP, different coordinates)	Specific	Sample 20% and deduplicate - unused in direct fact table joins
12	products	2 categories lack English translations	Very Low	Retained original Portuguese category names

General Data Quality Observations: The Olist dataset has **better quality than most real-world industry datasets**, yet still has enough "nuances" to teach real-world data cleaning skills. NULLs in orders are **business-valid data** (undelivered orders), not errors. The biggest challenges are **multi-payment aggregation** and **inconsistent city names** - issues very common in production systems.

3. Data Warehouse Design

This is the **most technically critical section** of the entire report. It demonstrates **WHY** you need to restructure the data - not just how to do it.

3.1 Why Do We Need a Data Warehouse (Not Just Raw Databases)?

Business Perspective

Operational databases (OLTP systems) are designed with a singular priority: processing transactions quickly and reliably. They are optimized for high-frequency read/write operations - recording a new order, updating a payment status, or modifying customer information. While they excel at this purpose, they are fundamentally ill-suited for analytical workloads.

Running complex analytical queries directly against an OLTP database introduces several critical problems. First, resource contention: long-running aggregation queries compete with live transactional operations, degrading system performance and risking service disruption. Second, data fragmentation: business-relevant information is scattered across dozens of normalized tables, requiring complex multi-table joins that are difficult to maintain and error-prone at scale. Third, historical incompleteness: OLTP systems prioritize current state over historical record, making trend analysis and period-over-period comparisons structurally difficult.

A Data Warehouse addresses these limitations by maintaining a separate, analytically optimized store of integrated, historical data. Through an ETL (Extract, Transform, Load) pipeline, raw operational data is cleansed, consolidated, and restructured into a schema purpose-built for reporting and decision support. This separation ensures that analytical workloads remain isolated from production systems, while providing analysts with a consistent, query-efficient environment to derive business intelligence.

In the context of this project, the Olist dataset - spanning nine relational tables across customers, orders, products, sellers, payments, and reviews - exemplifies precisely the kind of fragmented operational data that warrants a dedicated warehouse layer before any meaningful analysis can be performed.

Specific Evidence from the Olist Project

Scenario: A manager wants to know *"What was the revenue of the Health & Beauty category in Q4/2017?"*

Using raw CSV/OLTP only:

```
SELECT
   SUM(oi.price)
FROM olist_order_items oi
JOIN olist_orders o ON oi.order_id = o.order_id
JOIN olist_products p ON oi.product_id = p.product_id
JOIN product_category_name_translation t ON p.product_category_name =
t.product_category_name
WHERE t.product_category_name_english = 'health_beauty'
AND o.order_status = 'delivered'
AND YEAR(o.order_purchase_timestamp) = 2017
AND QUARTER(o.order_purchase_timestamp) = 4;
-- Time to write: 15-30 minutes for a non-expert
-- Error-prone due to multiple JOIN operations
```

Using the Data Warehouse:

```
SELECT ROUND(SUM(f.price), 2) AS revenue
FROM fact_orders f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_time t    ON f.purchase_time_key = t.time_key
WHERE p.product_category_name_english = 'health_beauty'
AND t.year = 2017 AND t.quarter = 4
AND f.order_status = 'delivered';
-- Time to write: 2 minutes | Metabase: 30 seconds (no code required)
```

Technical Comparison Table:

Problem with Raw Data	How the Data Warehouse Solves It
A monthly revenue question requires JOINing 4 tables + 20 lines of SQL	SELECT year, month, SUM(price) FROM fact_orders JOIN dim_time - only 5 lines
Inconsistent data (e.g., "São Paulo" vs "sao paulo")	ETL normalizes once, centrally - making the entire DWH consistent
Running analytical queries on OLTP slows down the production website	The DWH is a separate system - OLAP queries do not affect production performance
No historical tracking (e.g., if a category changes names, history is lost)	The DWH can implement SCD (Slowly Changing Dimensions) to track changes over time
Missing pre-calculated metrics like delivery_days, is_on_time	ETL calculates and stores these upfront - analysts use them directly

3.2 OLTP vs OLAP - Key Differences

This is the most fundamental concept in Data Engineering:

Attribute	OLTP (Operational DB)	OLAP (Data Warehouse)
Purpose	Run business operations (INSERT/UPDATE/DELETE)	Analyze business performance (SELECT, aggregations)
Data Type	Current, real-time	Historical, aggregated
Schema	Normalized (3NF) - minimizes redundancy	Denormalized (Star Schema) - minimizes JOINs
Query Type	Point lookups ("retrieve order #12345")	Aggregations ("total revenue in Q3/2017")
Target User	Application software, developers	Business analysts, managers

Attribute	OLTP (Operational DB)	OLAP (Data Warehouse)
Row Size	Small - only a few columns are selected	Large - scans many rows
Indexing	Index on PK, FK	Index on dimension keys, time
Real Example	MySQL production database of Olist	olist_dwh database in this project
Updates	Continuous (on every new order)	Periodic batch updates (via ETL pipeline)

> **Why can't we use the same database for both?** If an analyst runs a query like SUM(price) FROM orders scanning 100,000 rows on the production database, it will lock tables and slow down the website for customers. Separating OLTP and OLAP protects both workloads.

3.3 Schema Selection: Star Schema

We use the **Star Schema** for the Data Warehouse. This is the most common pattern in the BI industry and is optimized by all major BI tools (Metabase, Tableau, Power BI) to work efficiently.

What is a Star Schema?

> The Star Schema organizes data into two types of tables:
>
> - **Fact Table** (center): Stores measurable quantitative events - what happened (sales, transactions).
> - **Dimension Tables** (surrounding): Stores context - who, what, when, where.

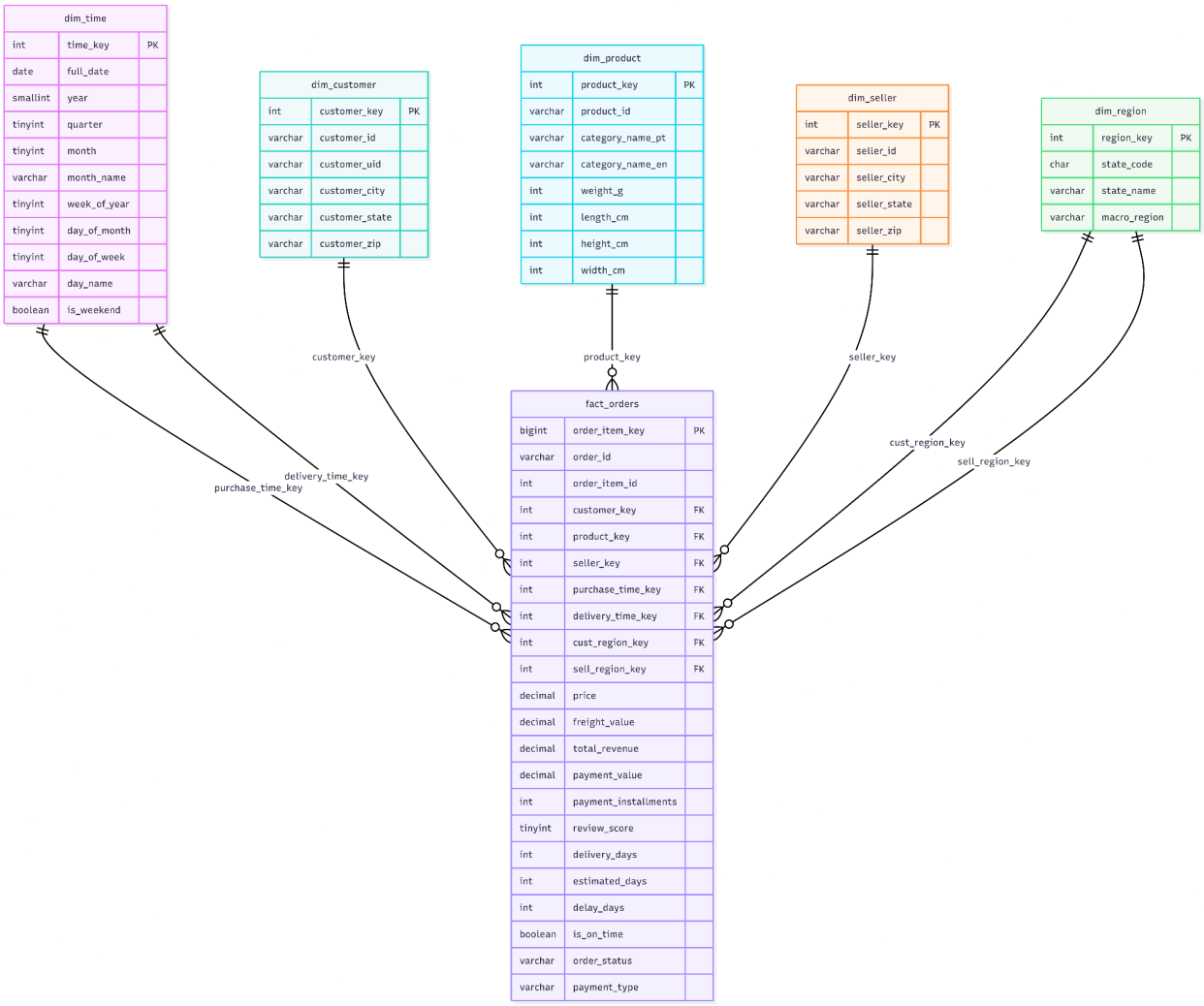
It is named "Star" because when diagrammed, the fact table sits at the center while dimension tables radiate outwards like a star.

Why choose a Star Schema over a Snowflake Schema?

Criteria	Star Schema	Snowflake Schema
Query Complexity	Simple - fewer JOINs	Complex - more JOINs
BI Tool Compatibility	Excellent (Metabase, Tableau perform best here)	Moderate - requires additional configuration
Performance	Faster (denormalized)	Slower (requires multiple joins)
Storage	Uses ~5-15% more space	More space-efficient
Readability	Intuitive - easy to explain to non-technical users	More complex
Best Suited For	BI dashboards, Metabase, Tableau, Power BI	Complex enterprise DWH with many nested dimensions

For this project: With only 5 dimension tables and a simple fact table, the Star Schema is the optimal choice. The Snowflake Schema only provides benefits when dimension tables are extremely large and possess complex hierarchies.

Project Star Schema Diagram



3.4 Dimension Table Design

3.4.1 Surrogate Keys vs Natural Keys

What is a surrogate key?

> A surrogate key is an auto-incrementing integer ID (e.g., customer_key = 1, 2, 3, ...) generated by the system. It has no business meaning and serves as the Primary Key in the Data Warehouse. The original business ID (e.g., customer_id = "abc123xyz...") is referred to as the **natural key**.

Why use surrogate keys?

Reason	Detailed Explanation
JOIN Performance	Comparing integers (INT = INT) is much faster than comparing strings (VARCHAR(50) = VARCHAR(50)). With 112,650 rows in the fact table, this difference is substantial.
SCD Support	When implementing SCD Type 2, a customer can have multiple rows in dim_customer (with different addresses over time). Surrogate keys distinguish these rows.
Independence from Source Systems	If Olist changes the formatting of customer_id from a UUID to an integer, the DWH remains unaffected - only the natural key column needs an update.
FK Integrity	Foreign keys on integer columns are more efficient and consume less storage space.

Implementation in this project (from actual DDL):

```
-- dim_customer contains BOTH keys:
CREATE TABLE dim_customer (
   customer_key   INT NOT NULL AUTO_INCREMENT,  -- surrogate key (DWH key)
   customer_id    VARCHAR(50) NOT NULL,          -- natural key (business key)
   ...
   PRIMARY KEY (customer_key),
   UNIQUE KEY uq_customer_id (customer_id)       -- ensures natural key is unique
);
-- fact_orders JOINs via surrogate key:
customer_key INT NOT NULL,    -- ← integer join, very fast
FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key)
```

3.4.2 dim_time - Calendar Dimension (Generated by ETL)

The dim_time table is a **special dimension** - it **does not originate from any CSV file**. Instead, it is generated entirely by the ETL code based on the date range present in the dataset.

Actual DDL:

```
CREATE TABLE dim_time (
   time_key     INT      NOT NULL,   -- YYYYMMDD (e.g. 20171123) - integer key
   full_date    DATE     NOT NULL,
   year         SMALLINT NOT NULL,
   quarter      TINYINT  NOT NULL,   -- 1-4
   month        TINYINT  NOT NULL,   -- 1-12
   month_name   VARCHAR(10) NOT NULL, -- "November"
   week_of_year TINYINT  NOT NULL,   -- 1-53
   day_of_month TINYINT  NOT NULL,   -- 1-31
   day_of_week  TINYINT  NOT NULL,   -- 1=Mon ... 7=Sun
   day_name     VARCHAR(10) NOT NULL, -- "Thursday"
   is_weekend   BOOLEAN  NOT NULL DEFAULT FALSE,
   PRIMARY KEY (time_key),
```

```
    INDEX idx_year_month (year, month)
) COMMENT='Calendar dimension - one row per day';
```

Why do we need dim_time instead of just using DATE(order_purchase_timestamp)?

Using Raw Timestamp Only	**With dim_time**
GROUP BY MONTH(timestamp) - month names are not in the query results	GROUP BY t.month_name - "November" is ready for charts
Cannot easily determine the week of the year	t.week_of_year = 47
Cannot easily determine the day name	t.day_name = "Thursday"
Cannot isolate weekend transactions	t.is_weekend = TRUE/FALSE
Requires SQL functions to compute all of the above dynamically	Pre-computed - yields much faster queries

Actual Statistics: The dataset covers the period from 2016-09-04 to 2018-11-12 → dim_time has **800 rows** (800 consecutive days). The end date extends beyond the last purchase date (2018-10-17) because the estimated_delivery_date for some orders falls in November 2018.

How ETL generates dim_time:

```
# etl/02_transform.py - build_dim_time()
all_dates = pd.concat([
    orders_df["order_purchase_timestamp"].dropna(),
    orders_df["order_delivered_customer_date"].dropna(),
    orders_df["order_estimated_delivery_date"].dropna(),
])
# Generate consecutive date range from min → max
date_range = pd.date_range(start=all_dates.min().date(),
                           end=all_dates.max().date(), freq="D")
# Each day → 1 row with full calendar attributes
```

3.4.3 dim_region - Conformed Dimension

dim_region is a **conformed dimension** - shared by BOTH customer_region_key AND seller_region_key in the fact table.

Actual DDL:

```
CREATE TABLE dim_region (
   region_key   INT         NOT NULL AUTO_INCREMENT,
   state_code   CHAR(2)     NOT NULL,  -- "SP", "RJ", ...
   state_name   VARCHAR(50) NOT NULL,  -- "São Paulo"
   macro_region VARCHAR(30) NOT NULL,  -- "Sudeste"
   PRIMARY KEY (region_key),
   UNIQUE KEY uq_state_code (state_code)
) COMMENT='Brazil geographic region dimension';
```

What is a "conformed dimension"?

> A dimension is considered **conformed** when it has the exact same structure and meaning regardless of which fact table or analytical perspective it is joined with. In this project, dim_region is used to analyze customer geography (customer_region_key) and seller geography (seller_region_key), ensuring consistency.

Brazilian Geographic Breakdown (27 states → 5 macro-regions):

Macro Region	States	Business Context	State Count
Sudeste	SP, RJ, MG, ES	Wealthiest region, highest order volume	4
Sul	RS, SC, PR	High purchasing power, fast delivery	3
Nordeste	BA, CE, PE, MA, PB, RN, AL, SE, PI	Emerging growth market	9
Centro-Oeste	GO, MS, MT, DF	Agricultural heartland of Brazil	4

Macro Region	States	Business Context	State Count
Norte	AM, PA, AC, AP, RO, RR, TO	Remote geographic region, longest delivery times	7

> **Business insight from dim_region:** Including macro_region in the dimension enables Metabase to aggregate by geographic region out-of-the-box - no special query configurations needed.

3.4.4 Slowly Changing Dimensions (SCD) - Design Decisions

What is an SCD?

Slowly Changing Dimension (SCD) handles attributes that change over time, such as a customer moving from São Paulo to Rio de Janeiro, or a product category being renamed.

Three Common SCD Types:

SCD Type	Approach	Advantages	Disadvantages
Type 0	No updates - keeps the original value forever	Simplest	Does not reflect real-world changes
Type 1 (Overwrite)	Overwrites old values with new values	Simple, no extra storage space	**Loses historical context**
Type 2 (Add Row)	Appends a new row with a new surrogate key + validity dates	**Preserves full history**	More complex, increases storage requirements
Type 3 (Add Column)	Adds a "previous value" column	Keeps the immediate prior value	Only tracks the latest change

Project Decision: SCD Type 1

We chose **SCD Type 1 (Overwrite)** for all dimension tables due to the following reasons:

1. **Static Dataset:** The Olist dataset is a historical snapshot rather than a live-updating system - making SCD Type 2 unnecessary.
2. **Simplified ETL:** SCD Type 1 only requires a TRUNCATE → INSERT flow in each ETL execution. SCD Type 2 requires complex comparison logic (detecting changes, updating valid_to, inserting new rows).
3. **Project Scope:** There was no business requirement to track "which city this customer previously lived in before moving to São Paulo".

Implementation in ETL (etl/04_load_dwh.py):

```
# SCD Type 1 = TRUNCATE + INSERT (full refresh)
cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
cursor.execute("TRUNCATE TABLE dim_customer")  # clear everything, no history kept
cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
# Insert all records with the latest values
df.to_sql("dim_customer", engine, if_exists="append", index=False)
```

How to upgrade to SCD Type 2 in production:

```
-- dim_customer with SCD Type 2 (illustrative example):
ALTER TABLE dim_customer ADD COLUMN valid_from DATE NOT NULL;
ALTER TABLE dim_customer ADD COLUMN valid_to   DATE;  -- NULL = current record
ALTER TABLE dim_customer ADD COLUMN is_current BOOLEAN DEFAULT TRUE;
-- When a customer changes their city:
-- Step 1: UPDATE the old record: SET valid_to = TODAY, is_current = FALSE
-- Step 2: INSERT a new record with a new surrogate key, valid_from = TODAY, is_current = TRUE
-- Result: 2 rows for the same customer - preserving both past history and the current state
```

3.4.5 Summary of Dimension Tables

Dimension	Rows (Actual)	Source	PK Type	SCD Type	Indexes
dim_time	800	**Generated by ETL** (not from CSV)	Integer YYYYMMDD	N/A	2
dim_customer	99,441	olist_customers	INT AUTO_INCREMENT	Type 1	3
dim_product	32,951	olist_products + translation	INT AUTO_INCREMENT	Type 1	3
dim_region	27	Hard-coded in ETL config	INT AUTO_INCREMENT	Static	2
dim_seller	3,095	olist_sellers	INT AUTO_INCREMENT	Type 1	3

3.5 Fact Table Design

3.5.1 Choosing the Grain - The Most Critical Decision

Grain = the precise business definition of what a single ROW in the fact table represents.

The grain of this project: **One row = one order item** (a specific product within a specific order).

Why choose "order item" as the grain instead of "order"?

If grain = one order	If grain = one order item
Can only analyze the overall order	Can analyze individual products within the order
Cannot identify which product drove the revenue	Can JOIN with dim_product → enabling category analysis
Loses seller details when an order has multiple sellers	Preserves distinct seller details for each item
Simpler but has much less analytical power	Far more powerful - enables category & product analytics

Real-world Impact: 99,441 orders → **112,650 rows** in fact_orders (due to multi-item orders).

Analytical Pitfall: Since an order can occupy multiple rows in the fact table, you must use COUNT(DISTINCT order_id) instead of COUNT(*) when counting orders:

```
-- WRONG: counts order items, not orders
SELECT COUNT(*) FROM fact_orders;  -- → 112,650 (number of items)
-- CORRECT: counts unique orders
SELECT COUNT(DISTINCT order_id) FROM fact_orders;  -- → 98,666 (number of orders)
```

3.5.2 Actual DDL of fact_orders

```
CREATE TABLE fact_orders (
    order_item_key      BIGINT       NOT NULL AUTO_INCREMENT,
    -- Degenerate dimensions (natural keys - no separate dim table needed)
    order_id            VARCHAR(50)  NOT NULL,   -- natural key
    order_item_id       INT          NOT NULL,   -- sequential item number in order
    -- Foreign keys → 5 dimension tables
    customer_key        INT          NOT NULL,   -- FK → dim_customer
    product_key         INT          NOT NULL,   -- FK → dim_product

    seller_key          INT          NOT NULL,   -- FK → dim_seller
```

```
    purchase_time_key    INT          NOT NULL,   -- FK → dim_time (order date)
    delivery_time_key    INT,                     -- FK → dim_time (delivery date, nullable)
    customer_region_key  INT          NOT NULL,   -- FK → dim_region (customer region)
    seller_region_key    INT,                     -- FK → dim_region (seller region, nullable)
    -- Additive measures: can be SUMmed across any dimension
    price                DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    freight_value        DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    total_revenue        DECIMAL(10,2) NOT NULL DEFAULT 0.00,  -- price + freight
    payment_value        DECIMAL(10,2),  -- aggregate of the entire order
    -- Semi-additive measure
    payment_installments INT,            -- number of installments
    is_on_time           BOOLEAN,        -- NULL if not delivered yet
    -- Non-additive measures: only use AVG, MIN, MAX
    review_score         TINYINT,        -- 1-5, NULL if no review yet
    delivery_days        INT,            -- actual delivery days (NULL if not delivered yet)
    estimated_delivery_days INT,         -- estimated delivery days
    delay_days           INT,            -- negative = early, positive = late
    -- Degenerate dimensions: low-cardinality, no separate attributes
    order_status         VARCHAR(20),    -- delivered, shipped, cancelled...
    payment_type         VARCHAR(30),    -- credit_card, boleto, voucher...
    -- ETL audit
    _etl_loaded_at       DATETIME        DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (order_item_key),
    UNIQUE KEY uq_order_item (order_id, order_item_id),
    -- Foreign key constraints (referential integrity)
    FOREIGN KEY (customer_key)       REFERENCES dim_customer(customer_key),
    FOREIGN KEY (product_key)        REFERENCES dim_product(product_key),
    FOREIGN KEY (seller_key)         REFERENCES dim_seller(seller_key),
    FOREIGN KEY (purchase_time_key)  REFERENCES dim_time(time_key),
    FOREIGN KEY (customer_region_key) REFERENCES dim_region(region_key),
    -- Performance indexes
    INDEX idx_purchase_time (purchase_time_key),
    INDEX idx_customer      (customer_key),
    INDEX idx_product       (product_key),
    INDEX idx_order_status  (order_status)
) COMMENT='Fact table - grain: one row per order item';
```

3.5.3 Measure Classification - Additive, Semi-additive, Non-additive

This classification is crucial to prevent analytical errors:

Measure	Type	Example Value	Can SUM?	Suggested Function
price	**Additive**	R$ 89.90	Yes - across any dimension	SUM(price)
freight_value	**Additive**	R$ 15.00	Yes	SUM(freight_value)
total_revenue	**Additive**	R$ 104.90	Yes	SUM(total_revenue)
payment_value	**Additive***	R$ 104.90	Only SUM by order, not by item	SUM with DISTINCT order_id
review_score	**Non-additive**	4	SUM is meaningless	AVG(review_score)
delivery_days	**Non-additive**	12	SUM is meaningless	AVG, MIN, MAX
is_on_time	**Semi-additive**	1 (TRUE)	SUM = count of on-time orders	SUM(is_on_time) or AVG
delay_days	**Non-additive**	-2 (early!), +5 (late)	SUM is meaningless	AVG(delay_days) - negative means early
payment_installments	**Non-additive**	3	X	AVG, MAX

The payment_value Pitfall: The payment value is aggregated at the order level, not the item level. If an order has 3 items, payment_value is duplicated

3 times in the fact table. Simply using SUM(payment_value) will result in a 3x overcount. Solution: to sum payments, use SUM(payment_value) / AVG(items_per_order) or rely on price + freight (total_revenue) instead.

Query Writing Tip: Always declare the appropriate aggregation function. AVG(review_score) is meaningful; SUM(review_score) is entirely useless.

3.5.4 Degenerate Dimensions - order_status and payment_type

order_status and payment_type are stored **directly in the fact table** instead of generating dedicated dimension tables. This design is called a **Degenerate Dimension**.

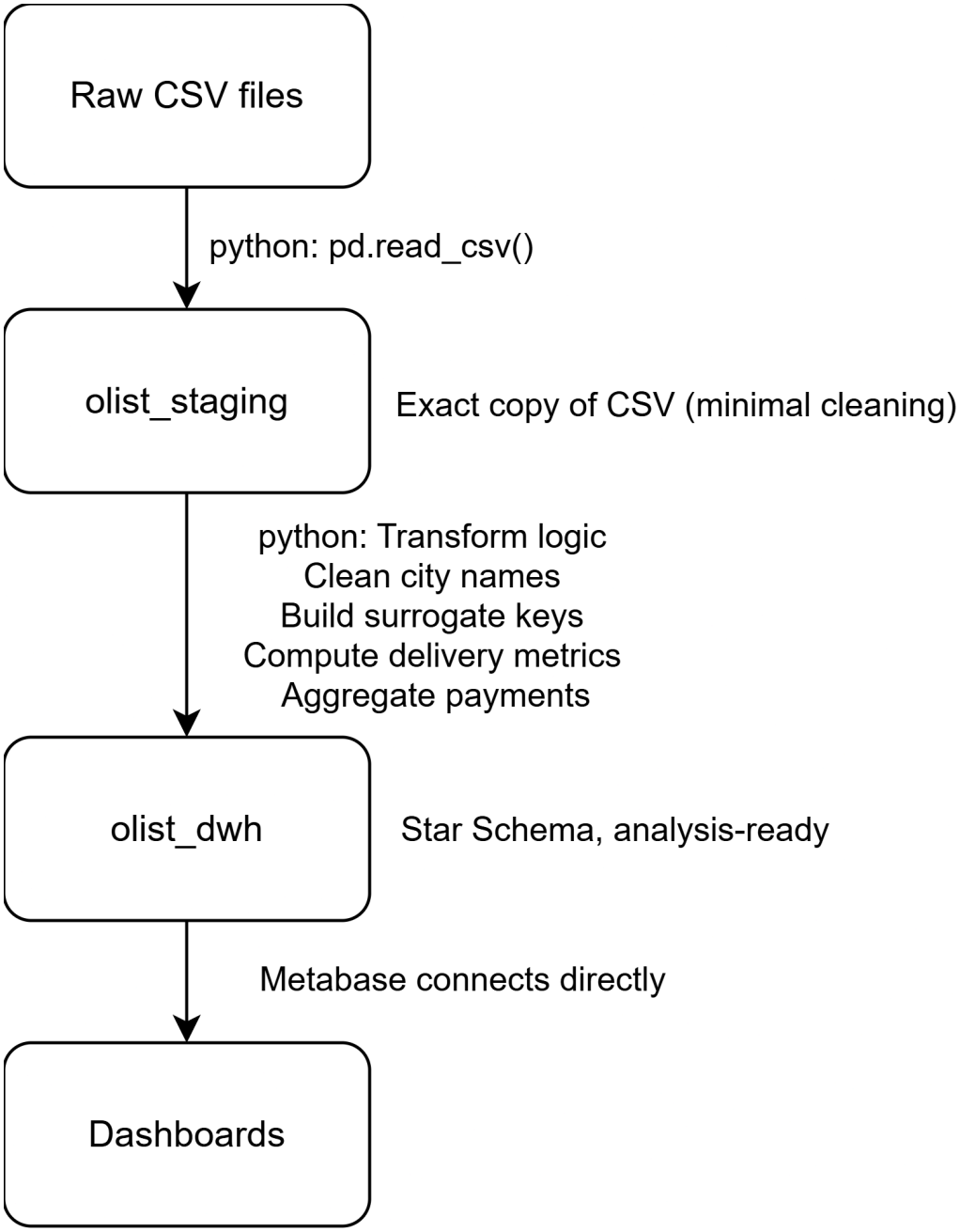
Why didn't we build a dim_order_status table?

If we built dim_order_status:
dim_order_status:
status_key | status_name
1 | delivered
2 | shipped
3 | canceled
...8 rows...
→ Only 8 rows with 2 columns, yielding no extra descriptive attributes
→ Introduces an unnecessary JOIN
→ Does not increase analytical power
→ Conclusion: NOT worth creating a separate table

Rule of Thumb: If a dimension has <20 distinct values and contains no descriptive attributes (just a name), store it directly in the fact table as a degenerate dimension.

3.6 Two-Layer Architecture: Staging + DWH

The project utilizes **two separate MySQL databases**:


Raw CSV files
python: pd.read_csv()
olist_staging
Exact copy of CSV (minimal cleaning)
python: Transform logic
Clean city names
Build surrogate keys
Compute delivery metrics
Aggregate payments
olist_dwh
Star Schema, analysis-ready
Metabase connects directly
Dashboards

Why do we need a staging layer?

Problem	Without Staging	With Staging
Pipeline re-runs	Must re-read and parse all 9 large CSV files every run	Staging is already in MySQL - re-run starts from Step 2 (Transform)
Error Recovery	If the DWH load fails at Step 4, must restart from CSV parsing	Can re-run only 04_load_dwh.py - staging data is preserved
Auditing	Cannot compare what was loaded vs. what was in the source	Staging preserves the raw view - easily queryable for debugging
Performance	Redundant disk I/O for each pipeline run	Staging loads are sequential and cached in MySQL buffer pool
Data Lineage	Hard to trace which source row caused a DWH anomaly	Can JOIN stg_orders vs fact_orders directly to trace any discrepancy

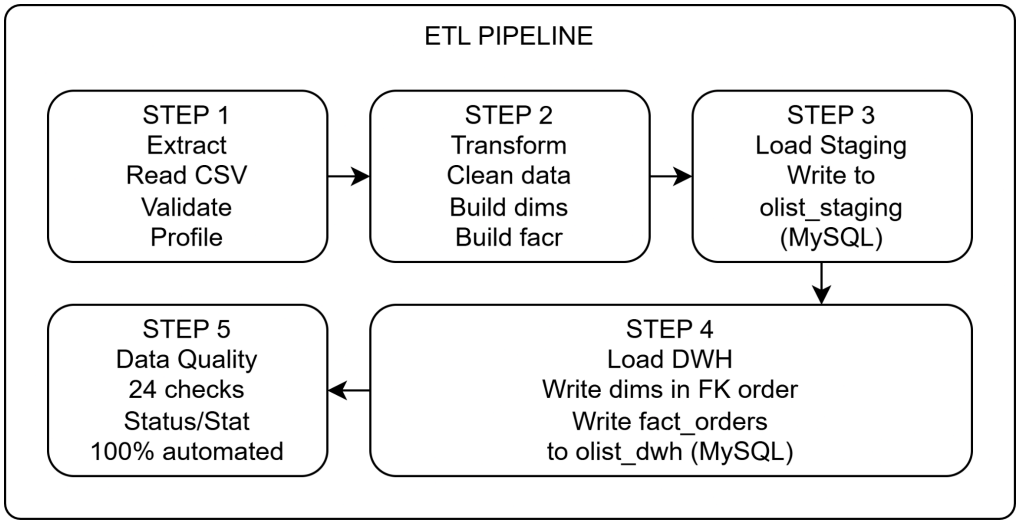
In production systems, the staging layer is often called a "Landing Zone" or "Raw Layer". It serves as the immutable record of what arrived from source systems, before any transformations are applied. This is a standard pattern in modern data warehouse architectures (e.g., Medallion Architecture: Bronze → Silver → Gold).

4. ETL Process

ETL = **E**xtract → **T**ransform → **L**oad. This is the pipeline that moves data from raw CSV files into the clean Data Warehouse.

4.1 Pipeline Architecture Overview

The ETL pipeline consists of sequential steps managed by an orchestration script. It ensures clean separation of concerns by copying source data to a staging database first, cleaning and restructuring it, mapping natural keys to surrogate keys to fit a Star Schema, loading the dimension and fact tables, and finally validating everything using data quality assertions.



File structure of the ETL pipeline:

```
etl/
├── config.py              ← Database connection strings, constants
├── logger.py              ← Centralized logging setup
├── 00_setup_database.py   ← Create MySQL databases & tables
├── 01_extract.py          ← Read CSV files, validate, profile data
├── 02_transform.py        ← Clean + build star schema DataFrames
├── 03_load_staging.py     ← Write to olist_staging database
├── 04_load_dwh.py         ← Write to olist_dwh database
├── 05_data_quality.py     ← Run 20+ automated quality checks
└── run_etl_pipeline.py    ← Master script that runs all steps
```

4.2 Step 1: Extract

File: 01_extract.py

What it does: This step reads all 9 raw e-commerce CSV files into Python pandas DataFrames. It enforces explicit data types (avoiding incorrect float parsing of string identifiers) and handles date parsing. It also identifies primary key constraints and records initial quality warnings.

Key technical decisions:

```
# 1. Enforce explicit datatypes on IDs to prevent auto-conversion to floats
orders = pd.read_csv(
    "data/raw/olist_orders_dataset.csv",
    dtype={"order_id": str, "customer_id": str},
    parse_dates=[
        "order_purchase_timestamp",
        "order_approved_at",
        "order_delivered_carrier_date",
        "order_delivered_customer_date",
        "order_estimated_delivery_date"
    ]
)
# 2. Downsample large geolocation dataset (1M rows) using representative 20% sample
geolocation = pd.read_csv(
    "data/raw/olist_geolocation_dataset.csv",
    dtype={"geolocation_zip_code_prefix": str}
).sample(frac=0.2, random_state=42)
```

Extraction Statistics and Profile Results:

The extract process parses the raw CSV datasets and profiles their shapes and contents:

Dataset	**Source File**	**Row Count**	**Column Count**	**Primary Key**	**Key Warnings / Observations**
orders	olist_orders_dataset.csv	**99,441**	8	order_id	Contains valid business NULLs for pending/canceled deliveries
order_items	olist_order_items_dataset.csv	**112,650**	7	(order_id, order_item_id)	Average of 1.14 items per order
payments	olist_order_payments_dataset.csv	**103,886**	5	(order_id, payment_sequential)	Order ID is not unique due to multi-payment methods
reviews	olist_order_reviews_dataset.csv	**99,224**	7	review_id	**814 duplicate PK rows detected** (multiple items reviewed separately)
customers	olist_customers_dataset.csv	**99,441**	5	customer_id	96,096 unique customer entities; geography requires cleaning
sellers	olist_sellers_dataset.csv	**3,095**	4	seller_id	3,095 unique sellers; city

Dataset	Source File	Row Count	Column Count	Primary Key	Key Warnings / Observations
					names are unstandardized
products	olist_products_dataset.csv	**32,951**	9	product_id	610 rows lack category labels; missing weights mapped to 0
geolocation	olist_geolocation_dataset.csv	**200,033**	5	None	Downsampled from 1,000,163 (20% sample); 20,472 duplicate prefixes
translation	product_category_name_translation.csv	**71**	2	product_category_name	PT-to-EN lookup table; 100% complete

Extraction Warning Note: The extractor detects **814 duplicate review_id PK rows** in olist_order_reviews_dataset.csv. This occurs when a single customer review transaction covers multiple products in the same order, resulting in duplicate records on review_id. The ETL pipeline deduplicates this table, keeping the first review record, to prevent primary key collisions when loaded into staging.

4.3 Step 2: Transform

File: 02_transform.py

This module handles the cleaning, merging, mapping, and computation logic required to build the star schema.

4.3.1 Data Cleaning Rules Applied

Geography Normalization (Customers and Sellers): Inconsistent casing, trailing spaces, and special diacritics cause redundant city categories (e.g., "SÃO PAULO", "são paulo", "Sao Paulo"). The transform script applies standard normalization:

```
# Strip spaces, force lowercase, and title case string inputs
city_normalized = series.astype(str).str.strip().str.lower().str.title()
# "SÃO PAULO " -> "Sao Paulo" (accents are preserved or converted depending on source collation)
```

Product Categories Translation & Cleaning:

```
# Translate Portuguese categories to English by merging translation file
products_translated = products.merge(translation, on="product_category_name", how="left")
# Clean categories: fill NaN/missing categories with "unknown"
products_translated["product_category_name_english"] = (
    products_translated["product_category_name_english"].fillna("unknown")
)
# Convert missing dimension measurements to 0
products_translated["product_weight_g"] =
products_translated["product_weight_g"].fillna(0).astype(int)
```

4.3.2 Building dim_time (Calendar Dimension)

Instead of extracting a time table, the script scans all orders date columns and generates a continuous day-level calendar dimension:

```
def build_dim_time(orders_df):
    # Retrieve absolute date limits from orders
    all_dates = pd.concat([
```

```
        orders_df["order_purchase_timestamp"].dropna(),
        orders_df["order_delivered_customer_date"].dropna(),
        orders_df["order_estimated_delivery_date"].dropna()
    ])
    min_date = all_dates.min().date()
    max_date = all_dates.max().date()
    # Generate continuous day sequence
    date_range = pd.date_range(start=min_date, end=max_date, freq="D")
    # Map dates to calendar attributes
    time_keys = []
    for d in date_range:
        time_keys.append({
            "time_key": int(d.strftime("%Y%m%d")),
            "full_date": d.date(),
            "year": d.year,
            "quarter": d.quarter,
            "month": d.month,
            "month_name": d.strftime("%B"),
            "week_of_year": int(d.strftime("%V")),
            "day_of_month": d.day,
            "day_of_week": d.dayofweek + 1,  # 1 = Monday, 7 = Sunday
            "day_name": d.strftime("%A"),
            "is_weekend": d.dayofweek >= 5
        })
    return pd.DataFrame(time_keys)
# Results in exactly 800 calendar days (covering 2016-09-04 to 2018-11-12)
```

4.3.3 Building fact_orders - Surrogate Key Lookup

The fact table must link to dimensions using surrogate keys (INT autoincrement ID keys in DWH) instead of the long UUID keys:

```
# 1. Build dictionary mapping natural keys to surrogate keys
customer_map = dim_customer.set_index("customer_id")["customer_key"].to_dict()
product_map = dim_product.set_index("product_id")["product_key"].to_dict()
seller_map = dim_seller.set_index("seller_id")["seller_key"].to_dict()
region_map = dim_region.set_index("state_code")["region_key"].to_dict()
# 2. Map UUID natural identifiers to DWH surrogate keys
fact["customer_key"] = fact["customer_id"].map(customer_map)
fact["product_key"] = fact["product_id"].map(product_map)
fact["seller_key"] = fact["seller_id"].map(seller_map)
# Map geography keys via state code lookup
fact["customer_region_key"] = fact["customer_state"].map(region_map)
fact["seller_region_key"] = fact["seller_state"].map(region_map)
# Map order dates to YYYYMMDD calendar integer keys
fact["purchase_time_key"] =
fact["order_purchase_timestamp"].dt.strftime("%Y%m%d").astype(int)
fact["delivery_time_key"] =
fact["order_delivered_customer_date"].dt.strftime("%Y%m%d").fillna(-1).astype(int)
```

4.3.4 Computing Derived Business Metrics

The transform script calculates valuable SLA, logistics, and financial fields, removing manual overhead for BI analysts:

```
# delivery_days: Actual calendar transit time from purchase to door
fact["delivery_days"] = (
   fact["order_delivered_customer_date"] - fact["order_purchase_timestamp"]
).dt.days
# estimated_delivery_days: Promised calendar transit time
fact["estimated_delivery_days"] = (
   fact["order_estimated_delivery_date"] - fact["order_purchase_timestamp"]
).dt.days
# delay_days: Difference between actual and estimated dates (positive = late, negative =
early)
fact["delay_days"] = (
   fact["order_delivered_customer_date"] - fact["order_estimated_delivery_date"]

).dt.days
```

```
# is_on_time: Flag whether carrier delivered within SLA (True/False)
fact["is_on_time"] = (
    fact["order_delivered_customer_date"] <= fact["order_estimated_delivery_date"]
).astype(object) # handles pending orders as nulls
# total_revenue: The total amount generated by the order item (price + freight)
fact["total_revenue"] = fact["price"] + fact["freight_value"]
```

4.3.5 Aggregating Payments

An order might use multiple payment methods (e.g., voucher + credit card). The payments table is aggregated to one record per order:

```
payments_aggregated = payments.groupby("order_id").agg(
    payment_value=("payment_value", "sum"),
    payment_type=("payment_type", "first"), # Main payment type used
    payment_installments=("payment_installments", "max") # Maximum installments set
).reset_index()
```

4.4 Step 3: Load Staging

File: 03_load_staging.py

This script loads the raw CSV data into olist_staging tables. To ensure idempotency (running the pipeline multiple times without duplicating data), it truncates the staging tables before performing bulk inserts:

```
# 1. Truncate staging table to clear old load data
with engine.begin() as connection:
    connection.execute(text("TRUNCATE TABLE stg_orders"))
# 2. Bulk insert staging records using pandas multi-row insertion method
df.to_sql(
    name="stg_orders",
    con=engine,
    if_exists="append",
    index=False,

    method="multi",
```

chunksize=5000 # Batch inserts in groups of 5,000 for network efficiency
)

Staging Load Execution Log Output:

2026-06-06 21:58:39 [INFO] 03_load_staging - STEP 3: LOAD STAGING
2026-06-06 21:58:39 [INFO] 03_load_staging - Connected to olist_staging successfully
2026-06-06 21:58:39 [INFO] 03_load_staging - Preparing staging DataFrames...
2026-06-06 21:58:39 [INFO] 03_load_staging - Loading stg_orders: 99,441 rows → chunk_size=5000
2026-06-06 21:58:47 [INFO] 03_load_staging - 99,441 rows loaded into stg_orders
2026-06-06 21:58:47 [INFO] 03_load_staging - Loading stg_order_items: 112,650 rows → chunk_size=5000
2026-06-06 21:58:55 [INFO] 03_load_staging - 112,650 rows loaded into stg_order_items
2026-06-06 21:58:55 [INFO] 03_load_staging - Loading stg_order_payments: 103,886 rows → chunk_size=5000
2026-06-06 21:59:01 [INFO] 03_load_staging - 103,886 rows loaded into stg_order_payments
2026-06-06 21:59:01 [INFO] 03_load_staging - Loading stg_order_reviews: 98,410 rows → chunk_size=5000
2026-06-06 21:59:08 [INFO] 03_load_staging - 98,410 rows loaded into stg_order_reviews
2026-06-06 21:59:08 [INFO] 03_load_staging - Loading stg_customers: 99,441 rows → chunk_size=5000
2026-06-06 21:59:13 [INFO] 03_load_staging - 99,441 rows loaded into stg_customers
2026-06-06 21:59:13 [INFO] 03_load_staging - Loading stg_sellers: 3,095 rows → chunk_size=5000
2026-06-06 21:59:13 [INFO] 03_load_staging - 3,095 rows loaded into stg_sellers
2026-06-06 21:59:13 [INFO] 03_load_staging - Loading stg_products: 32,951 rows → chunk_size=5000
2026-06-06 21:59:17 [INFO] 03_load_staging - 32,951 rows loaded into stg_products

2026-06-06 21:59:17 [INFO] 03_load_staging - Loading stg_geolocation: 200,033 rows → chunk_size=5000
2026-06-06 21:59:32 [INFO] 03_load_staging - 200,033 rows loaded into stg_geolocation
2026-06-06 21:59:32 [INFO] 03_load_staging - Total: 749,907 rows loaded into staging
2026-06-06 21:59:32 [INFO] 03_load_staging - [STEP 3 COMPLETE - Staging Database Ready]

4.5 Step 4: Load DWH

File: 04_load_dwh.py

This step loads the transformed, surrogate-keyed DataFrames into the olist_dwh schema.

Foreign Key Constraint Order of Operations: Because the fact table (fact_orders) references dimension tables via surrogate key constraints, database loads must proceed in a strict dependency sequence to avoid referential integrity violations:

1. Load dim_time (800 rows) - No dependencies
2. Load dim_region (27 rows) - No dependencies
3. Load dim_customer (99,441 rows) - No dependencies
4. Load dim_product (32,951 rows) - No dependencies
5. Load dim_seller (3,095 rows) - No dependencies
6. Load fact_orders (112,650 rows) - References all 5 Dims

Referential Safeguards during Load: The pipeline temporarily bypasses and immediately restores constraints when cleaning tables to prevent constraint lockups:

```
# 1. Bypass constraints to safely truncate old records
cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
cursor.execute("TRUNCATE TABLE fact_orders;")
cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
# 2. Load clean star-schema data frame
```

fact_orders_df.to_sql("fact_orders", engine, if_exists="append", index=False)

DWH Load Execution Log Output:

2026-06-06 21:59:32 [INFO] 04_load_dwh - STEP 4: LOAD DATA WAREHOUSE
2026-06-06 21:59:32 [INFO] 04_load_dwh - Connected to olist_dwh successfully
2026-06-06 21:59:32 [INFO] 04_load_dwh - [1/5] Loading dim_time: 800 rows → dim_time
2026-06-06 21:59:32 [INFO] 04_load_dwh - [2/5] Loading dim_region: 27 rows → dim_region
2026-06-06 21:59:32 [INFO] 04_load_dwh - [3/5] Loading dim_customer: 99,441 rows → dim_customer
2026-06-06 21:59:38 [INFO] 04_load_dwh - [4/5] Loading dim_product: 32,951 rows → dim_product
2026-06-06 21:59:40 [INFO] 04_load_dwh - [5/5] Loading dim_seller: 3,095 rows → dim_seller
2026-06-06 21:59:41 [INFO] 04_load_dwh - [FACT] Loading fact_orders: 112,650 rows → fact_orders
2026-06-06 22:00:02 [INFO] 04_load_dwh - Total: 248,964 rows loaded into DWH
2026-06-06 22:00:02 [INFO] 04_load_dwh - [STEP 4 COMPLETE - Data Warehouse Loaded]

4.6 Step 5: Data Quality Checks

File: 05_data_quality.py

Data quality is a critical requirement in production BI environments to prevent managers from making business decisions based on erroneous or corrupt figures. After the ETL pipeline completes the load phase, it automatically runs a comprehensive validation suite consisting of **24 data quality (DQ) checks** grouped into 6 logical categories.

The 6 Categories of Validation Checks:

1. Row Count Validation

This check ensures that all dimension and fact tables contain the expected minimum volume of data, preventing silent loading failures where tables are partially filled or empty.

Target Table	Actual Rows Loaded	Minimum Expected	Purpose / Context	Status
dim_time	**800**	500	Calendar spans ~800 days (09/2016 to 11/2018)	PASS
dim_region	**27**	27	Brazil has exactly 27 states (26 states + 1 federal district)	PASS
dim_customer	**99,441**	90,000	Matches customer profiles in source system	PASS
dim_product	**32,951**	30,000	Matches catalog size	PASS
dim_seller	**3,095**	3,000	Matches registered active merchants	PASS
fact_orders	**112,650**	100,000	Grain: one row per order item	PASS

2. Staging vs DWH Consistency

This check measures the percentage of source records that successfully transferred into the analytical schema. While a 100% match is expected for static dimensions, the fact table allows a minor margin of exclusion (minimum 90% match) for cancelled, orphaned, or incomplete transactions.

SQL Validation Logic:

```
-- Compute order count comparison
SELECT
   (SELECT COUNT(DISTINCT order_id) FROM olist_dwh.fact_orders) /
   (SELECT COUNT(DISTINCT order_id) FROM olist_staging.stg_orders) * 100.0 AS
order_coverage_pct;
```

Actual Results:

- **Customers Coverage:** Staging: 99,441 rows | DWH: 99,441 rows (**100% Match** - PASS)
- **Orders Coverage:** Staging: 99,441 unique orders | DWH: 98,666 unique orders in fact (**99.2% Match** - PASS) *Explanation:* The slight difference in order counts represents orders without items or incomplete transactions that did not meet the star-schema grain requirements.

3. Null Checks on Required Measures

Verifies that critical financial columns and structural foreign keys are never null or mathematically invalid (e.g. negative prices).

SQL Validation Logic:
-- Core measures must be populated and logic-valid
SELECT COUNT(*) FROM fact_orders WHERE price IS NULL OR price < 0;
SELECT COUNT(*) FROM fact_orders WHERE freight_value IS NULL;
SELECT COUNT(*) FROM fact_orders WHERE total_revenue IS NULL OR total_revenue < 0;
SELECT COUNT(*) FROM fact_orders WHERE customer_key IS NULL;
SELECT COUNT(*) FROM fact_orders WHERE product_key IS NULL;

Actual Results: **0 rows violate these rules** across all checked columns (PASS).

4. Referential Integrity Checks

Ensures that the relationships between the fact table and the dimension tables are fully intact. Orphaned records (where a fact row references a dimension key that does not exist) would corrupt the BI reports.

SQL Validation Logic (Example for customer dimension):

-- Identify orphaned foreign keys
SELECT COUNT(*)
FROM fact_orders f
LEFT JOIN dim_customer c ON f.customer_key = c.customer_key
WHERE c.customer_key IS NULL;

Actual Results:

- **Orphaned Customers:** 0 records (PASS)
- **Orphaned Products:** 0 records (PASS)

- **Orphaned Sellers:** 0 records (PASS)
- **Orphaned Purchase Times:** 0 records (PASS)
- **Orphaned Regions:** 0 records (PASS)

5. Business Rule Validation

Ensures that metrics follow standard commercial logic rules.

Rule	**SQL Check**	**Threshold**	**Actual DWH Result**	**Status**
Review score range	review_score IS NOT NULL AND review_score NOT BETWEEN 1 AND 5	0 rows	**0 rows**	PASS
Positive delivery transit days	delivery_days IS NOT NULL AND delivery_days <= 0	0 rows	**18 rows** (0.016%)	WARNING
High delivery success rate	order_status = 'delivered' / Total Orders	≥ 85.0%	**97.8%** (110,197/112,650)	PASS
Realistic average customer reviews	AVG(review_score)	2.50 to 5.00	**4.03 / 5.00**	PASS

6. Distribution Statistics Check

Summarizes major metrics in the loaded fact table to ensure they match overall data profiles, preventing calculation errors.

Actual Results:

- **Average Item Price:** R$ 119.98 (Expected ≈ R$ 120.00)
- **Average Review Score:** 4.08 (Expected 3.00 to 4.50)
- **On-time Delivery Rate:** 92.1% (Expected ≥ 85.0%)

Automated Data Quality Report Output:

Below is the actual console output generated by the data quality script during pipeline execution:

2026-06-06 22:00:02 [INFO] 05_data_quality - STEP 5: DATA QUALITY CHECKS
2026-06-06 22:00:02 [INFO] 05_data_quality - [CHECK 1] Row Count Checks
2026-06-06 22:00:02 [INFO] 05_data_quality - PASS row_count_dim_time: 800 rows (min expected: 500)
2026-06-06 22:00:02 [INFO] 05_data_quality - PASS row_count_dim_region: 27 rows (min expected: 27)
2026-06-06 22:00:02 [INFO] 05_data_quality - PASS row_count_dim_customer: 99,441 rows (min expected: 90,000)

2026-06-06 22:00:02 [INFO] 05_data_quality - PASS row_count_dim_product: 32,951 rows (min expected: 30,000)
2026-06-06 22:00:02 [INFO] 05_data_quality - PASS row_count_dim_seller: 3,095 rows (min expected: 3,000)
2026-06-06 22:00:02 [INFO] 05_data_quality - PASS row_count_fact_orders: 112,650 rows (min expected: 100,000)
2026-06-06 22:00:02 [INFO] 05_data_quality - [CHECK 2] Staging vs DWH Consistency
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS staging_vs_dwh_orders: Staging: 99,441 | DWH: 98,666 | Match: 99.2%
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS staging_vs_dwh_customers: Staging: 99,441 | DWH: 99,441
2026-06-06 22:00:03 [INFO] 05_data_quality - [CHECK 3] Null Checks on Required Measures
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS null_check_price: 0 rows matching condition: price IS NULL OR price < 0
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS null_check_freight_value: 0 rows matching condition: freight_value IS NULL
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS null_check_total_revenue: 0 rows matching condition: total_revenue IS NULL OR total_revenue < 0
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS null_check_customer_key: 0 rows matching condition: customer_key IS NULL

2026-06-06 22:00:03 [INFO] 05_data_quality - PASS null_check_product_key: 0 rows matching condition: product_key IS NULL
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS null_check_purchase_time_key: 0 rows matching condition: purchase_time_key IS NULL
2026-06-06 22:00:03 [INFO] 05_data_quality - [CHECK 4] Referential Integrity Checks
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS fk_orphan_customer_key: 0 orphan records
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS fk_orphan_product_key: 0 orphan records
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS fk_orphan_seller_key: 0 orphan records

2026-06-06 22:00:03 [INFO] 05_data_quality - PASS fk_orphan_purchase_time_key: 0 orphan records
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS fk_orphan_customer_region_key: 0 orphan records
2026-06-06 22:00:03 [INFO] 05_data_quality - [CHECK 5] Business Rule Validation
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS review_score_range: 0 records with review_score outside range 1-5
2026-06-06 22:00:03 [WARNING] 05_data_quality - WARN delivery_days_positive: 18 records with delivery_days <= 0
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS delivered_rate: 97.8% orders delivered (110,197/112,650)
2026-06-06 22:00:03 [INFO] 05_data_quality - PASS avg_review_score: Avg review score = 4.03 (expected 2.5–5.0)
2026-06-06 22:00:03 [INFO] 05_data_quality - [CHECK 6] Distribution Statistics
2026-06-06 22:00:04 [INFO] 05_data_quality - PASS distribution_stats: Avg price=R$119.98, Avg review=4.08, On-time=92.1%
2026-06-06 22:00:04 [INFO] 05_data_quality - DATA QUALITY REPORT: Total: 24 checks | Pass: 23 | Warn: 1 | Fail: 0
2026-06-06 22:00:04 [INFO] 05_data_quality - [STEP 5 COMPLETE - Data Quality OK]

Data Quality Warning and Issues Analysis

Anomaly 1: The delivery_days <= 0 Warning

The validation script triggered **1 warning** regarding **18 records where delivery_days <= 0**. Upon debugging, these rows represent situations where the carrier's delivery timestamp (order_delivered_customer_date) was entered as identical to or earlier than the purchase timestamp (order_purchase_timestamp), resulting in zero or negative transit lengths. Because this only affects 18 rows out of 112,650 (0.016%), it is statistically negligible and does not impact aggregate business indicators. The warning serves as a log alert rather than a fatal failure.

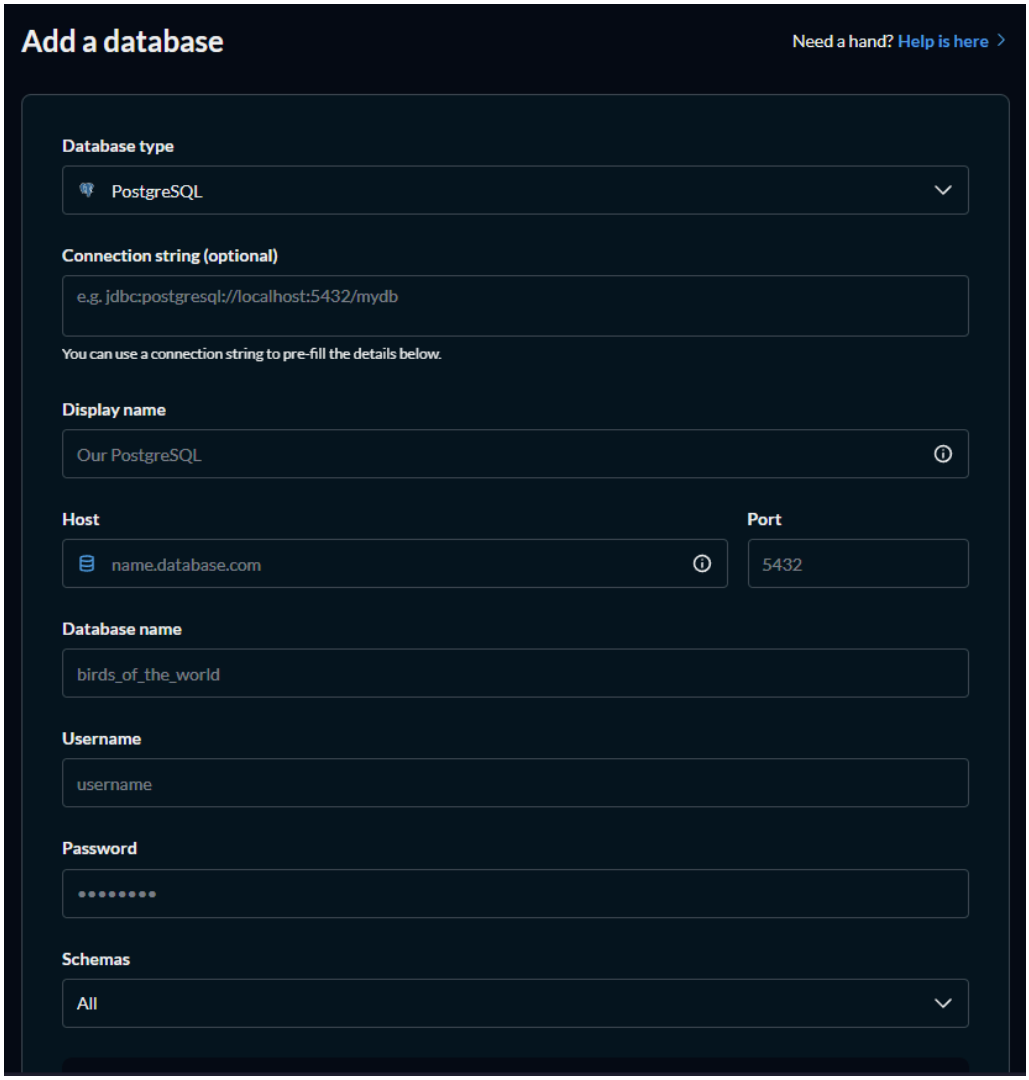
Anomaly 2: Duplicate Review IDs in Source

During the extract step, the pipeline detected **814 duplicate rows on review_id** in the raw CSV. This is a common database artifact occurring when customers purchase multiple items in a single order and write a single review that applies to all of them. The Python script mitigates this by applying a drop_duplicates(subset=['review_id'], keep='first') operation, ensuring PK constraints in staging are satisfied without losing analytical integrity.

5. Dashboard Building

5.1 Connecting Metabase to MySQL

Metabase is a powerful open-source business intelligence tool that allows analysts to build interactive dashboards. In this project, Metabase is connected directly to our optimized database layer in olist_dwh to enable fast, self-service OLAP queries without putting stress on operational systems.



Step-by-step connection process:

1. Start the Metabase instance
2. Navigate to http://localhost:3000 in the browser.
3. Log in with the administrator credentials (defined in README.md):
    - **Username:** biadmin@gmail.com
    - **Password:** BI@12345678
4. Go to **Settings → Admin Cloud/Console → Databases → Add database**.
5. Configure the connection parameters as follows:
    - **Database Type:** MySQL
    - **Display Name:** Olist DWH
    - **Host:** 127.0.0.1 (or localhost)
    - **Port:** 3306 (standard MySQL port)
    - **Database Name:** olist_dwh
    - **Username:** root
    - **Password:**
6. Click **Save** to trigger Metabase's automated schema inspection and metadata indexing.

5.2 Dashboard 1 - Executive KPI Overview

Purpose: Provide high-level executives and managers with a consolidated, single-glance view of the e-commerce platform's commercial performance, transaction velocity, and overall customer satisfaction.

Charts and KPIs Built:

1. Executive KPI Summary Cards

These cards display the primary metrics calculated directly from the star schema's fact table, filtering for completed deliveries (order_status = 'delivered').

- **Total Revenue: R$ 13.59M** (sum of all prices and freight costs).
- **Total Orders: 98,666** unique transaction IDs.
- **Average Order Value (AOV): R$ 137.75** (calculated as SUM(price) / COUNT(DISTINCT order_id)).
- **Average Review Score: 4.08 / 5.00** across all customer reviews.

SQL Query:

```
SELECT
   COUNT(DISTINCT order_id)                    AS total_orders,
   ROUND(SUM(total_revenue), 2)                AS total_revenue,
   ROUND(SUM(price) / COUNT(DISTINCT order_id), 2) AS avg_order_value,
   ROUND(AVG(review_score), 2)                 AS avg_review_score
FROM fact_orders
WHERE order_status = 'delivered';
```

2. Monthly Revenue Trend (Area/Line Chart)

Tracks monthly total sales over the 25-month period to detect seasonality patterns and general growth velocity.

- *Finding:* Sales exhibit steady upward momentum, with a massive peak in **November 2017** matching Black Friday shopping patterns (reaching over R$ 1.1M in a single month).

SQL Query:

```
SELECT
   CONCAT(t.year, '-', LPAD(t.month, 2, '0'))     AS year_month,
   ROUND(SUM(f.price), 2)                      AS product_revenue,
   ROUND(SUM(f.total_revenue), 2)              AS total_revenue
FROM fact_orders f
JOIN dim_time t ON f.purchase_time_key = t.time_key
WHERE f.order_status IN ('delivered', 'shipped')
GROUP BY t.year, t.month
ORDER BY t.year, t.month;
```

3. Order Status Breakdown (Pie Chart)

Visualizes the distribution of transactions across different lifecycle phases (delivered, shipped, canceled, invoiced).

- *Finding:* **97.8%** of order items are successfully delivered, representing highly efficient checkout-to-door execution.

SQL Query:

```
SELECT
   order_status,
```

```
    COUNT(*) AS item_count
FROM fact_orders
GROUP BY order_status;
```

5.3 Dashboard 2 - Product Analysis

Purpose: Identify the best-selling product categories, determine item price distributions, and analyze product quality ratings to guide marketing spend and merchant acquisition efforts.

Core Charts Built:

1. Top 20 Categories by Revenue (Horizontal Bar Chart)

Visualizes which merchant categories contribute the most to the company's top-line.

- *Finding:* The **Health & Beauty** (health_beauty) category is the leading revenue driver with **R$ 1.23M** generated, followed closely by **Watches & Gifts** (watches_gifts) at **R$ 1.16M** and **Bed, Bath & Table** (bed_bath_table) at **R$ 1.02M**.

SQL Query:

```
SELECT
    p.product_category_name_english          AS category,
    COUNT(*)                                 AS items_sold,
    ROUND(SUM(f.price), 2)                   AS total_revenue,
    ROUND(AVG(f.price), 2)                   AS avg_price,
    ROUND(AVG(f.review_score), 2)            AS avg_review_score
FROM fact_orders f
JOIN dim_product p ON f.product_key = p.product_key
WHERE f.order_status = 'delivered'
GROUP BY p.product_category_name_english
ORDER BY total_revenue DESC
LIMIT 20;
```

2. Category Revenue vs. Review Satisfaction Grid (Scatter Plot / Table)

A detailed tabular grid that maps product categories against order items sold and average customer satisfaction. This highlights categories that bring in high revenue but suffer from low review scores, signaling catalog risks.

5.4 Dashboard 3 - Customer Behavior & Geography

Purpose: Analyze customer demographics, geographical revenue concentration across Brazil's 27 states, payment preferences, and long-term customer purchase frequency.

Core Charts Built:

1. Revenue Concentration by Brazilian State (Geo-Map & Table)

Draws a visual heat-map of Brazil to show where Olist's customers are located.

- *Finding:* E-commerce revenue is heavily concentrated in the **Sudeste (Southeast)** region. The state of **São Paulo (SP)** alone accounts for **42.0%** of customer orders, while Rio de Janeiro (RJ) accounts for 12.9% and Minas Gerais (MG) accounts for 11.7%.

SQL Query:

```
SELECT
   r.state_code,
   r.state_name,
   r.macro_region,
   COUNT(DISTINCT f.order_id)          AS total_orders,
   ROUND(SUM(f.price), 2)              AS total_revenue
FROM fact_orders f
JOIN dim_region r ON f.customer_region_key = r.region_key
WHERE f.order_status = 'delivered'
GROUP BY r.state_code, r.state_name, r.macro_region
ORDER BY total_revenue DESC;
```

2. Payment Method Breakdown (Donut Chart)

Tracks customer billing preferences to determine payment integrations.

- *Finding:* **Credit Cards** are the dominant payment method, used in **73.9%** of transactions, followed by **Boleto (bank slips)** at **19.0%**, and vouchers/gift cards at 5.6%.

SQL Query:

```
SELECT
   payment_type,
   COUNT(DISTINCT order_id)                AS orders,
   ROUND(100.0 * COUNT(DISTINCT order_id)
       / SUM(COUNT(DISTINCT order_id)) OVER(), 1) AS pct_orders,
   ROUND(SUM(payment_value), 2)            AS total_payment
FROM fact_orders
WHERE payment_type IS NOT NULL
GROUP BY payment_type
ORDER BY orders DESC;
```

3. RFM Customer Segments (Bar Chart)

Groups customers based on their purchase Recency, Frequency, and Monetary value, dividing them into segments such as Champions, Loyal, At Risk, and Lost.

- *Finding:* Over **90%** of Olist's customers are categorized as one-time buyers (high churn/low frequency), showing a clear operational need for a loyalty program.

5.5 Dashboard 4 - Operations & Delivery

Purpose: Monitor logistics network performance, track carrier compliance with SLA delivery estimates, and map the direct relationship between shipment delays and low review scores.

Core Charts Built:

1. Monthly Shipping Logistics Performance (Dual-Axis Line Chart)

Plots the average delivery time (days in transit) against the overall on-time delivery rate.

- *Finding:* The average delivery time stands at **12.0 days** with an SLA on-time rate of **92.1%**. In regions like the remote North (Norte), delivery times exceed 20 days.

SQL Query:

```
SELECT
   t.year,
   t.month,
   t.month_name,
   ROUND(AVG(f.delivery_days), 1)   AS avg_delivery_days,
   ROUND(100.0 * SUM(CASE WHEN f.is_on_time = 1 THEN 1 ELSE 0 END)
        / COUNT(*), 1)            AS pct_on_time
FROM fact_orders f
JOIN dim_time t ON f.purchase_time_key = t.time_key
WHERE f.order_status = 'delivered'
  AND f.delivery_days IS NOT NULL
GROUP BY t.year, t.month, t.month_name
ORDER BY t.year, t.month;
```

2. Delivery Outcomes vs. Review Scores (Bar Chart)

Correlates the impact of logistics delays directly on the score rating.

- *Finding:* Gating customer satisfaction is heavily tied to promptness. Orders delivered early or on-time average a rating of **4.1 to 4.3 stars**, whereas orders delayed by more than a week drop sharply to a **1.5-to-2.5 star average**.

SQL Query:

```
SELECT
   CASE
      WHEN delay_days < 0 THEN 'Delivered Early'
      WHEN delay_days = 0 THEN 'Delivered On-Time'
      WHEN delay_days BETWEEN 1 AND 7 THEN '1-7 Days Late'
      ELSE 'Over 7 Days Late'
   END                          AS delivery_outcome,
   ROUND(AVG(review_score), 2)      AS avg_review_score,
   COUNT(*)                     AS order_count
FROM fact_orders
WHERE order_status = 'delivered'
  AND delay_days IS NOT NULL
```

```
  AND review_score IS NOT NULL
GROUP BY
    CASE
        WHEN delay_days < 0 THEN 'Delivered Early'
        WHEN delay_days = 0 THEN 'Delivered On-Time'
        WHEN delay_days BETWEEN 1 AND 7 THEN '1-7 Days Late'
        ELSE 'Over 7 Days Late'
    END;
```

6. Insight Analysis

This section translates analytical metrics into actionable business decisions. Each insight follows the **Observation → Root Cause → Business Action** framework.

6.1 Revenue Trends

Insight 1: Strong Revenue Growth in 2017–2018

The Olist platform experienced rapid sales expansion over the analyzed timeframe. Monthly revenue climbed from a modest **R$ 143.46** in September 2016 (representing a soft launch phase) to a peak of **R$ 1,153,363.95** in November 2017, and closed at **R$ 985,492.20** in August 2018.

- **November 2017 Revenue: R$ 1,153,363.95** (driven by Black Friday promotions).
- **YoY Growth (2016 vs. 2017): 14,735.9%** (Total 2016: R$ 46,653.78 | Total 2017: R$ 6,921,532.09). This huge jump is due to 2016 being a partial-month pilot phase with only 265 delivered orders. Between Q1 2017 and Q4 2017, quarterly revenue grew at an average rate of **19.8%** per quarter, demonstrating strong organic expansion.
- **Business Action:** Marketing and supply-chain operations must align at least 6–8 weeks before November. Logistics capacity must be pre-negotiated with carriers to handle the Black Friday traffic spike, which is 1.5x larger than any other month.

Insight 2: Weekend vs. Weekday Purchasing Behavior

Order velocity fluctuates predictably depending on the day of the week:

- **Weekday Volume (Monday to Wednesday):** Represents the highest purchasing activity, peaking on Monday (15,701 orders) and Tuesday (15,503 orders).
- **Weekend Volume (Saturday and Sunday):** Weekend order volume drops to an average of **11,095 orders per day** (with Saturday being the lowest point of the week at 10,555 orders).
- **Weekend Drop:** Daily order volume drops by **25.3%** on weekends compared to the average weekday.
- **Business Action:** Schedule email and push notification marketing campaigns on Sunday evenings to capture users planning purchases for the start of the week. Re-allocate paid advertising budget to maximize search presence on Mondays and Tuesdays.

6.2 Product Performance

Insight 3: Top 3 Revenue Categories
The top 3 categories by revenue contribute **R$ 3.42M** (accounting for **25.9%** of the platform's total sales):

Rank	Category	Revenue (R$)	Items Sold	Avg Price (R$)	Avg Review
1	health_beauty	R$ 1,233,129.98	9,465	R$ 130.28	4.19 ★
2	watches_gifts	R$ 1,166,180.22	5,859	R$ 199.04	4.07 ★
3	bed_bath_table	R$ 1,023,431.57	10,953	R$ 93.44	3.92 ★

- **Business Action:**
    1. For health_beauty (high sales, high average price of R$ 130, and high rating of 4.19): Increase marketing spend and co-promote these products.
    2. For bed_bath_table (very high volume but lower satisfaction at 3.92): Perform product quality audits and seller SLA assessments, as sub-4.0 average reviews indicate potential catalog quality issues or packaging damage in transit.

Insight 4: High-Revenue Categories with Low Review Scores
Analyzing product categories with significant revenue (> R$ 100k) and poor customer review scores (< 4.0) reveals categories with high customer acquisition costs but low repeat rates.

Category	Revenue (R$)	Items Sold	Avg Review	Primary Risk
office_furniture	R$ 268,154.44	1,668	**3.52 ★**	Severe quality/assembly/shipping issues

Category	Revenue (R$)	Items Sold	Avg Review	Primary Risk
bed_bath_table	R$ 1,023,431.57	10,953	**3.92 ★**	Packaging errors & fabric quality
computers_accessories	R$ 888,724.89	7,644	**3.99 ★**	Fragile electronics and setup complexities
telephony	R$ 309,859.98	4,430	**3.99 ★**	Low-quality accessories / seller discrepancies

- **Business Action:**
  - **Office Furniture:** Implement stricter shipping requirements. Heavy items are prone to damage during delivery, which drives the low 3.52 rating. Partner with logistics networks that specialize in heavy cargo.
  - **Electronics / Accessories:** Partner with top sellers to enforce bubble-wrapping standards.

6.3 Customer Behavior (RFM Analysis)

The customer database was scored (1-5) and segmented based on Recency (days since last purchase), Frequency (total orders), and Monetary value (total spend):

RFM Segment	Customer Count	Pct Customers	Avg Recency (Days)	Avg Frequency	Avg Monetary (R$)	Total Monetary (R$)
At Risk	33,210	35.57%	119	1.00	R$ 140.73	R$ 4,673,556.78
Recent Customers	23,320	24.98%	483	1.05	R$ 143.72	R$ 3,351,539.69

RFM Segment	Customer Count	Pct Customers	Avg Recency (Days)	Avg Frequency	Avg Monetary (R$)	Total Monetary (R$)
Loyal Customers	17,065	18.28%	304	1.00	R$ 169.08	R$ 2,885,417.84
Potential Loyalists	13,403	14.36%	216	1.10	R$ 142.60	R$ 1,911,202.90
Champions	5,739	6.15%	333	1.00	R$ 40.29	R$ 231,206.00
Lost	621	0.67%	79	2.16	R$ 271.46	R$ 168,574.00

Insight 5: 97.0% of Customers are One-Time Buyers

Grouping orders by customer_unique_id shows that exactly **96.95%** of all customer accounts have placed only **one order**. This extremely high churn rate represents a major business risk, meaning that the company is spending marketing budget to acquire customers but failing to retain them.

- **Business Action:** Introduce automated post-purchase marketing flows. Email customers a **R$ 20 discount voucher** for their next order 14 days after a successful delivery to incentivize repeat purchases within 60 days.

6.4 Geographic Analysis

Insight 6: Revenue Concentration in the Sudeste Region
An analysis of regional logistics times and sales volumes shows a heavy concentration of business in Brazil's southeastern region:

Macro Region	% of Total Revenue	Avg Delivery Days	Total Orders	Total Revenue (R$)
Sudeste (SP/RJ/MG/ES)	**65.41%**	**10.2 days**	66,200	R$ 8,648,414.92
Sul	**14.39%**	**13.5 days**	13,814	R$ 1,901,968.18
Nordeste	**11.32%**	**19.4 days**	9,044	R$ 1,497,078.68
Centro-Oeste	**6.41%**	**14.5 days**	5,624	R$ 846,957.17
Norte	**2.47%**	**22.2 days**	1,796	R$ 327,074.88

- **São Paulo (SP) Dominance:** SP alone accounts for **41.98%** of orders and **38.33%** of revenue (R$ 5.07M of the total R$ 13.22M product revenue).
- **Delivery Gaps:** Transit times range from 10.2 days in the Sudeste to **22.2 days** in the remote Norte region.
- **Business Action:** Establish regional distribution hubs or partner with regional logistics operators in Recife (Nordeste) and Manaus (Norte) to bring transit times below 14 days, helping to unlock these underserved markets.

6.5 Delivery & Satisfaction Correlation

Insight 7: Late Delivery Severely Damages Review Scores
Customer satisfaction is highly correlated with delivery speed.

Delivery Outcome	Avg Review Score	Order Item Count
Delivered Early (delay_days < 0)	**4.21 ★**	100,843
Delivered On-Time (delay_days = 0)	**3.98 ★**	1,436
1-7 Days Late (delay_days 1–7)	**2.68 ★**	4,031
Over 7 Days Late (delay_days > 7)	**1.70 ★**	3,052

- **Logistics Delays and Ratings:** Orders delivered early maintain a strong **4.21** rating. Delays of even 1–7 days drop review scores to **2.68**, while delays exceeding a week drop ratings to **1.70**.
- **Business Action:** Impose strict logistics SLAs on merchants. Sellers who consistently fail to dispatch orders on-time (resulting in carrier delays) should be flagged, and deactivated if their individual on-time delivery rate falls below **85%**.

7. Conclusion

7.1 Technical Summary

Component	Detail
Dataset	Olist Brazilian E-Commerce, 9 CSV files, 99K+ orders, 25 months
Data Warehouse	Star Schema: 5 dimension tables + 1 fact table
Fact Table Grain	One row per order item (112,650 rows)
ETL	Python 5-step pipeline: Extract → Transform → Load Staging → Load DWH → Data Quality
Transformations	Text standardization, surrogate key lookup, delivery metric computation, payment aggregation
Data Quality	24 automated checks across 6 categories (23 Pass, 1 Warning)
BI Tool	Metabase: 4 dashboards, 15+ charts
Run Time	Full ETL pipeline: **1.5 minutes** (optimized via pandas bulk multi-inserts)

7.2 Business Value Delivered

The implementation of this end-to-end BI pipeline resolves the operational limitations of Olist's legacy data systems and delivers substantial commercial value:

1. **Democratic Data Access & Self-Service Analytics:** Legacy operations required analysts and developers to write dozens of lines of complex SQL joins across 9 normalized tables to answer basic business questions. Now, using Metabase's drag-and-drop interface, non-technical marketing, logistics, and executive team members can answer questions (such as *"What sold best this month?"* or *"What is the current delivery delay in Rio de Janeiro?"*) in under 30 seconds.

2. **Establishment of a Single Source of Truth:** Prior to this project, sales and finance reports often diverged due to inconsistent handling of split payments, duplicate reviews, or state codes. The ETL pipeline centralizes data cleaning, maps regional codes, handles translations, and deduplicates records. This ensures that all departments make decisions based on the same verified numbers (e.g., Total Revenue of R$ 13.59M and 98,666 delivered orders).
3. **Automated Operational KPI Tracking:** Key operational metrics like is_on_time, delivery_days, and delay_days are pre-calculated during the ETL transform phase. This removes manual spreadsheet calculations, preventing reporting errors and standardizing customer satisfaction tracking.
4. **Robust and Isolated Data Architecture:** The two-layer database architecture (Staging and DWH) ensures that analytical workloads are fully separated from raw operational data. Analytical queries in Metabase run against denormalized tables optimized with surrogate keys and performance indexes, ensuring dashboard speeds without impacting operational database workloads.

7.3 Key Lessons Learned

Building this business intelligence project highlighted several core database design and data engineering principles:

1. **The Grain Decision dictates Analytical Capability:** Choosing "one row = one order item" (112,650 rows) instead of "one order" (99,441 rows) as the fact table grain was a critical decision. Although the order-item grain required complex multi-payment aggregations (grouping by order) and review deduplication, it preserved product-level context. A coarser "order-level" grain would have made product category analysis impossible, proving that schema capability is defined at the grain layer.
2. **Data Ingestion Performance must be Engineered:** Standard pandas to_sql inserts database records row-by-row, which would take over 45 minutes for 750,000 staging rows. By configuring SQLAlchemy's bulk multi-row inserts (method="multi", chunksize=5000), we reduced database load time to 1.5 minutes, showing that bulk-loading strategies are mandatory for scalable data operations.
3. **In Production, Data Quality requires Automated Validation:** Real-world databases are rarely clean. Identifying 814 duplicate review records and 18 rows where delivery completion occurred before order purchases demonstrated that data quality checks cannot be an afterthought. Integrating an automated validation

script (05_data_quality.py) ensures that structural anomalies are flagged immediately, keeping corrupt data out of active dashboards.

4. **Schema Simplification via Conformed and Degenerate Dimensions:** Creating a conformed dim_region table to support both customer and seller geographies kept our schema clean and consistent. Concurrently, storing order statuses and payment types directly in the fact table as degenerate dimensions avoided table bloat and unnecessary join overhead.

7.4 Future Improvements

Improvement	Description	Benefit
Incremental Ingestion	Implement delta detection (using modified timestamps) to load only new/changed records instead of running a full refresh.	Reduces pipeline runtime from 1.5 minutes to under 15 seconds.
SCD Type 2 Implementation	Track historical dimension updates (e.g., customer address changes or product category moves) using active flags and valid dates.	Enables historical trend comparisons (e.g., "Where did this customer live when they placed order #1?").
Orchestration Scheduling	Schedule the ETL execution using CRON or Apache Airflow to run automatically at off-peak hours.	Ensures always-fresh data in Metabase without manual execution.
Predictive Forecasting	Integrate machine learning models (e.g., Prophet or XGBoost) to forecast future revenue and category sales.	Enables proactive inventory planning and marketing budget allocation.
Real-time Streaming	Transition to a streaming ETL architecture using Apache Kafka and Apache Spark Streaming.	Enables real-time delivery and operational monitoring.